Knowledge Representation

Knowledge

familiarity, awareness, or understanding of someone or something
(facts, rules)

Awareness or understanding of a circumstance or fact gained through association or experience. The human ability resulting from interpreted information

Understanding that germinates from combination of data, information, experience, and individual interpretation.

Knowledge

Having concepts, procedures, rules, ideas, facts, and associations, coupled with an ability to use these notions effectively in modeling different aspects of the world.

Knowledge may be procedural OR declarative.

Procedural knowledge is compiled knowledge related to the programmer e.g., the steps used to solve an algebraic equation are expressed as procedural knowledge.

Declarative knowledge, or passive knowledge is expressed as statement of facts about the world. Personal data in a database is typical of declarative knowledge. Such data are pieces of independent knowledge.

What is Knowledge Representation?
Knowledge Representation is the process of representing real-world knowledge in a form that a computer can understand and use for reasoning. Most fundamentally it is a surrogate used to enable an entity to determine consequences by thinking rather than acting, i.e., by reasoning about the world rather than taking action in it.

It is a medium for pragmatically efficient computation, i.e., the computational environment in which thinking is accomplished. One contribution to this pragmatic efficiency is supplied by the guidance a representation provides for organizing information so as to facilitate making the recommended inferences.

Knowledge representation is a multidisciplinary subject that applies theories and techniques from other fields:

1. **Logic** provides the formal structure and rules of inference.
2. **Ontology** defines the kinds of things that exist in the application domain.

Logic is a formal language for representing facts and properties of a world in a precise, unambiguous way.

Propositional Logic
Predicate Logic

Logic is concerned with reasoning and the validity of arguments. It is not concerned with the truth of the sentence, just their validity.

A piece of reason is valid if it leads to a true conclusion in every situation where the premises are true.

All men are mortal
Jack Sparrow is a man
Therefore, Jack Sparrow is mortal.

This set of statement is considered to be valid because the conclusion follows logically from the other two statements (premises)

All students love exams
You are a student
Therefore, you love exams

Propositional logic is not the study of truth, but of the relationship between the truth of one statement and that of another (Hedman, 2004)

Logic is defined by the following:
Syntax - describes the possible configurations that constitute sentences.
Semantics - determines what facts in the world the sentences refer to i.e. the interpretation. Each sentence makes a claim about the world.
Inference Methods - set of rules for generating new sentences that are necessarily true given that the old sentences are true. The relationship between sentences is called **entailment**. The semantics link these sentences (representation) to facts of the world. The proof can be used to determine new facts which follow from the old.

Propositional Logic
The simplest logic of all. Allows facts about the world to be represented as sentences formed from:

Propositional symbols: P, Q, R, S...
Not: ¬
And (^)
Or: (˅)
Implies: → (if)
iff (If and only if): ↔

Wrapping parentheses: (...)
logical constants: *true*, *false*, *unknown*
Quantifiers ∃, ∀

An expression is referred to as a ***well-formed-formula*** **(wff)** or a sentence if it is constructed correctly, according to the rules of the syntax of propositional calculus.

P ^ Q ⋎ (B ^ ¬C) → A ^ B ⋎ D ^ (¬E)

Propositional Logic

Atomic Propositions: simple propositions represented by a single proposition symbol
either true or false.

Earth is round
It is a hot day
Ice is cold

A := Earth is round
B := It is a hot day
C := Ice is cold

Compound proposition: Compound propositions are constructed by combining simpler or atomic propositions, using parenthesis and logical connectives.
To use propositional logic, it is first necessary to convert facts and rules about the real world into logical expressions using the logical operators
Let us examine some examples considering simple operators.

Sentences that use the word 'and' in English to express more than one concept, all of which is true at once, can be easily translated into logic using the AND operator, **Λ**. For example:

R:= It is Raining
F:= It is Friday

It is raining **and** it is Friday might be expressed as:

R **Λ** F

A:= It is hot
B:= The sky is cloudy
A ∧ B
A → B
C:= John can play Tennis
¬ C John cannot play tennis

John can play tennis or football

D:= John can play football
C ᵛ D

If John can play tennis then he can play football
C → D

If you live in France, you can speak French
A:= Live in France
B:= Speak French
A → B

If it is hot and humid, then it is raining

H:= It is hot
M:= It is humid
R:= It is raining
H ∧ M
H ∧ M → R

Propositional Logic is a useful tool for reasoning, but it has limitation because it cannot see inside prepositions and take advantage of relationships among them.

Predicate Logic
Predicate logic (predicate calculus or First-order logic) is a powerful language that develops information about the objects in an easier way and can also express the relationship between those objects.

Predicate logic allows us to reason about properties of objects and the relationships between objects.

Constants (John, London, Tennis)
Predicates (father, likes, …)
Variables (X,Y)
Quantifier (∃, ∀)

In predicate calculus, we use predicates to express properties of objects.

John is David's father

In order to represent a relationship between individual objects, we can use a predicate specifying the objects as its arguments

John is David's **father**
Father(John, David)

John **likes** London
Likes(John, London)

Likes(John, football)
Likes(John, photography)

Likes(John,**?**) (what does John like?)

Likes(John, London)
Likes(John, football)
Likes(John, photography)
Likes(Alice, tennis)
Likes(Adam, cricket)
Likes(Sam, tennis)
Likes(David, photography)

Question: What does John like?
Likes(John,**?**)

Question: Who likes photography?
Likes(**?**, photography)

- Alison likes Rachel and chocolate
likes(Alison, Rachel)
likes(Alison, Chocolate)

If Rachel and Alison are friends then Alison likes Rachel

Objects: Rachel and Alison
Predicates: friend and like

friend(Alison, Rachel) → **likes**(Alison, Rachel)
First-order logic
Universally quantified variables, e.g., ∀ X
Universal
for all, for each, for every

everyone, everything

All men ….

∀X: man(X)

Existentially quantified variables, e.g., ∃ X

Existential

for some, for at least one

There exists some

Some men …

∃ Y: man(Y)

Using variables with predicates to capture generalizations

We can capture generalizations by asserting that any instance of a given class has the relevant property. For example:

Every human is mortal

∀x: human(x) → mortal(x)

Some boys play cricket

predicate = play(x, y), where x = boy and y = game

∃x: boy(x) ^ play(x, cricket)

Some fruits are poisonous

∃ x: fruit(x) ^ poisonous(x)

Using quantifiers and variables

Quantification is used to distinguish general and specific assertions.

There is a white alligator:

∃x: alligator(x) ^ color(x,white)

There exists some bird that doesn't fly:

∃x: bird(x) ^ ¬ fly(x)

Every person has something that they love:

∀x: person(x) → ∃y: loves(x,y)

Some gardeners hate the rain.

∃x: gardener(x) ^ hates(x,rain)

More Examples

You can fool some people all of the time.

$\exists x: (person(x) \wedge \forall t: (time(t) \rightarrow canfool(x,t)))$

You can fool all of the people some of the time.

$\forall x: (person(x) \wedge \exists t: (time(t) \rightarrow canfool(x,t)))$

All purple mushrooms are poisonous.

$\forall x: (mushroom(x) \wedge color(x,purple) \rightarrow poisonous(x)$

No purple mushroom is poisonous.

$\forall x: (mushroom(x) \wedge color(x,purple) \rightarrow \neg poisonous(x)$

Every child loves icecream.

$\forall x: child(x) \rightarrow loves(x,icecream)$

Jane has at least one friend who is generous.

$\exists x: friend(Jane,x) \wedge generous(x)$

Everyone is a friend of someone.

$\forall x \exists y: friend(x,y)$

People only criticize people that are not their friends

$\forall x \forall y: (person(x) \wedge person(y) \wedge criticize(x,y) \rightarrow \neg friend(x,y))$

Predicate Logic - Resolution

Resolution is a theorem proving technique that works by building **refutation proofs**, i.e., **proofs by contradictions**.

It was invented by a Mathematician John Alan Robinson in the year 1965.

Resolution can resolve two clauses if they contain complementary literals.

Unification is a key concept in proofs by resolutions

Unification

Unification: a process of making two different logical atomic expressions identical by finding a substitution (matching literals)

Substitution: when a variable name is replaced by another variable or element of the domain.

likes(x,y)

likes(John, photography) assume/substitute x=John , y=photography

Two formulas are said to unify if there are possible **assignments of terms to variables** that make the formulas in question identical.

Resolution and CNF
Resolution is a single rule of inference that can operate efficiently on a special form of sentences.

The special form is called Conjunctive Normal Form (CNF) or clausal form, and has the following properties:

Every individual sentence is a disjunction (OR) of literals
Sentence1: likes(a,b) **V** plays(x,cricket)
Sentence2: eats(z,pasta) **V** loves(z,reading)

These individual sentences are implicitly conjuncted (ANDed)
Sentence1 **^** Sentence2
(likes(a,b) ∨ plays(x,cricket)) **^** (eats(z,pasta) ∨ loves(z,reading))

Conversion to CNF
Eliminate all implications →
Reduce the scope of all ¬ to single term
Make all variable names unique
Eliminate Existential Quantifiers
Eliminate Universal Quantifiers
Convert to conjunction of disjuncts
Create separate clause for each conjunct.

Exercise:
Harry, Ron and Draco are students.
Every student is either wicked or plays Quidditch.
No one who plays Quidditch likes rain.
Everyone wicked likes potions.
Draco dislikes whatever Harry likes and likes whatever Harry dislikes.
Draco likes rain and potions.

Is Draco wicked?

Steps for Resolution:
1. Conversion of facts into first-order logic.
2. Convert first-order logic statements into CNF
3. Negate the statement which needs to be proved
4. Draw resolution graph.

Convert to First-order Logic and Convert to CNF

Harry, Ron and Draco are students.

student(Harry)
student(Ron)
student(Draco)

Every student is either wicked or plays Quidditch.

$\forall x$: student(x) $\rightarrow$ wicked(x) $\vee$ plays(x,Quidditch)

CNF: A $\rightarrow$ B = $\neg$ A $\vee$ B

$\neg$ student(x) $\vee$ wicked(x) $\vee$ plays(x,Quidditch)

No one who plays Quidditch likes the rain

$\forall x$: plays(x,Quidditch) $\rightarrow$ $\neg$ likes(x,rain)

CNF: A $\rightarrow$ B = $\neg$ A $\vee$ B

$\neg$ plays(x,Quidditch) $\vee$ $\neg$ likes(x,rain)

Everyone wicked likes potions.

$\forall x$: wicked(x) $\rightarrow$ likes(x,potions)
$\neg$ wicked(x) $\vee$ likes(x,potions)

Draco dislikes whatever Harry likes and likes whatever Harry dislikes.

$\forall x$: likes(Harry,x) $\rightarrow$ $\neg$ likes(Draco,x)
$\forall x$: $\neg$ likes(Harry,x) $\rightarrow$ likes(Draco,x)

$\neg$ likes(Harry,x) $\vee$ $\neg$ likes(Draco,x)
likes(Harry,x) $\vee$ likes(Draco,x)

Draco likes rain and potions.

likes(Draco,rain)
likes(Draco,potions)

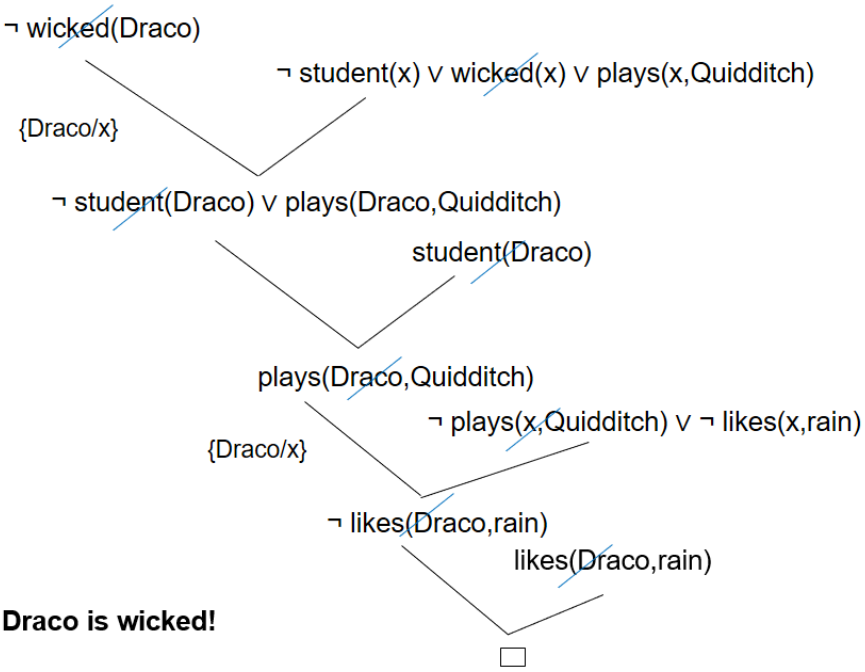
Negate the statement which needs to be proved

Is Draco wicked?
wicked(Draco)
$\neg$ wicked(Draco)

Resolution

1. student(Harry)
2. student(Ron)
3. student(Draco)
4. ¬ student(x) ∨ wicked(x) ∨ plays(x,Quidditch)
5. ¬ plays(x,Quidditch) ∨ ¬ likes(x,rain)
6. ¬ wicked(x) ∨ likes(x,potions)
7. ¬ likes(Harry,x) ∨ ¬ likes(Draco,x)
8. likes(Harry,x) ∨ likes(Draco,x)
9. likes(Draco,rain)
10. likes(Draco,potions)

Q: ¬ wicked(Draco)



Yes, Draco is wicked!

Apply Resolution Proof Procedure:
Every child loves Ironman.
Everyone who loves Ironman loves Spiderman.
Spiderman is a superhero.
Venom does not love Spiderman.

Prove: Venom is not a child.

Step1: Convert to First-order Logic
Every child loves Ironman.
$\forall x$: child(x) $\rightarrow$ loves(x,Ironman)

Everyone who loves Ironman loves Spiderman.
$\forall x$: loves(x, Ironman) $\rightarrow$ loves(x,Spiderman)

Spiderman is a superhero
superhero(Spiderman)

Venom does not love Spiderman
$\neg$ loves(Venom,Spiderman)

Step2: Convert to CNF
1. $\forall x$: child(x) $\rightarrow$ loves(x, Ironman)
   $\neg$ child(x) $\vee$ loves(x,Ironman)

2. $\forall x$: loves(x, Ironman) $\rightarrow$ loves(x,Spiderman)
   $\neg$ loves(x, Ironman) $\vee$ loves(x,Spiderman)

3. superhero(Spiderman)

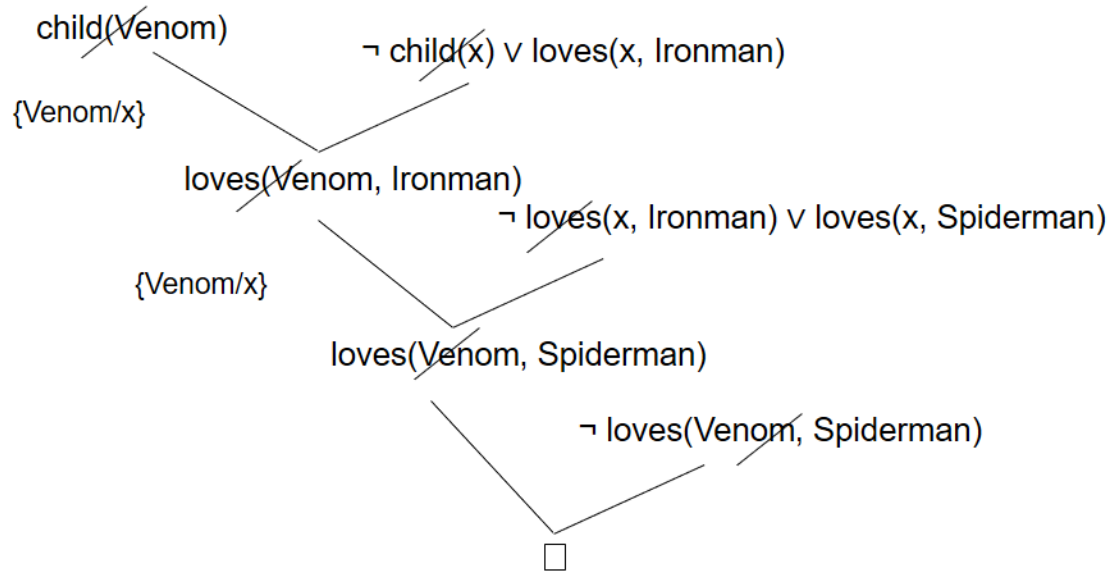
4. $\neg$ loves(Venom,Spiderman)

Step3: Negate the statement you need to prove

Venom is not a child.
$\neg$ child(Venom)

Negation:
child(Venom)

Step4: Draw resolution graph



True, Venom is not a child!

Resolution Graph
Apply Resolution Proof Procedure:
Everyone passing the test is happy.
Everyone who practices something can pass it.
John practices everything.
Everyone who is lucky can pass anything.
Sam is not lucky but practices everything.

Prove: John can pass the test.

Step1: Convert to FOL
Everyone passing the test is happy.
∀x: pass(x,test) → happy(x)

Everyone who practices something can pass it.
∀x ∃y : practice(x,y) → pass(x,y)

John practices everything.
∀x: practice(John,x)

Everyone who is lucky can pass anything.
∀x ∀y : lucky(x) → pass(x,y)

Sam is not lucky but he practices everything.

∀x: ¬lucky(Sam) ^ practice(Sam,x)

Step2: Convert to CNF

1. ∀x: pass(x,test) → happy(x)
¬ pass(x,test) ∨ happy(x)

2. ∀x ∃y : practice(x,y) → pass(x,y)
¬ practice(x,y) ∨ pass(x,y)

3. practice(John,x)

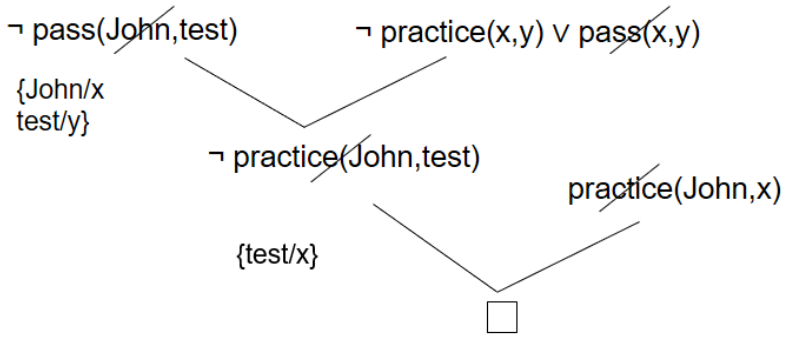
4. ∀x ∀y : lucky(x) → pass(x,y)
¬ lucky(x) ∨ pass(x,y)

5. ∀x: ¬lucky(Sam) ^ practice(Sam,x)
¬lucky(Sam)
practice(Sam,x)

Step3: Negate the statement you need to prove

John can pass the test.
pass(John,test)

Negation:
¬ pass(John,test)



True, John can pass the test!